

QCon

全球软件开发大会

ZillizTown

基于 Milvus 与 Claude Code 打造企业版 OpenClaw

陈彪

Zilliz 工程总监

QCon 2026 北京 · Agentic Engineering

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

- 01** 入口在迁移,软件在改变
- 02** ZillizTown 初见 · 4 个真实场景
- 03** 技术选型与架构
- 04** 拆解 ZillizTown 内部机制

01

入口在迁移，软件在改变

流量入口正在迁移

2025-2026: 12 个月集中爆发

2025.03	AWS	CLI Agent 发布
2025.08	Sentry	MCP Server 发布
2025.09	Cloudflare	首个远程 MCP Server
2025.10	Snowflake	MCP Server Preview
2025.10	Stripe	MCP + Agent Toolkit
2025.11	Vercel	MCP + Agent Tools
2025.12	Databricks	MCP 50+ 工具
2026.03	Google	Workspace CLI + MCP
2026.03	飞书	Lark CLI 开源, 200+ 命令

30,000,000

Sentry MCP 月请求量

以前

用户通过浏览器和 App 使用软件

现在

用户通过 Agent 使用 Skills

aws

SENTRY

CLOUDFLARE

snowflake

stripe

Vercel



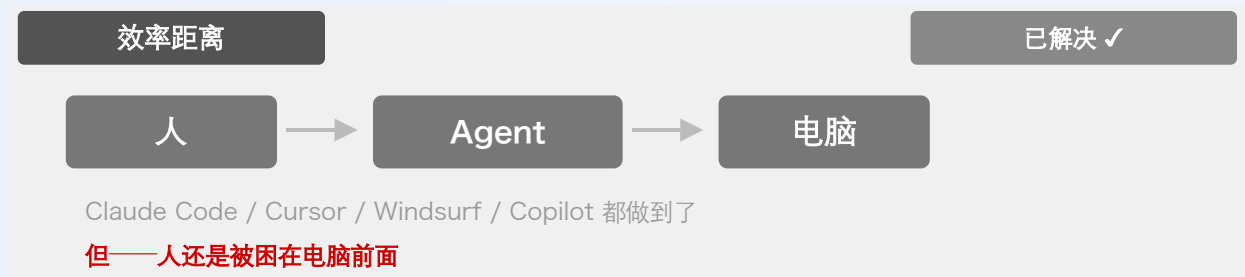
Google Workspace

它们不是在做 Agent, 而是在为 Agent 做接口

不适配 Agent 的软件, 就像 2010 年没做移动端

OpenClaw 多做了哪一步？

人与 Agent 之间的两段距离



Coding Agent 对比

Agent	效率距离	物理距离
Claude Code	✓	✗ 终端
Cursor	✓	✗ IDE
Windsurf	✓	✗ IDE
Copilot	✓	✗ IDE/终端
OpenClaw	✓	✓ IM 接入
ZillizTown	✓	✓ 飞书

335K Stars

核心原因不是功能多
是触达方式变了

效率距离所有人都在做，物理距离才是 OpenClaw 爆火的原因
这也是我们把展示层建在飞书上的原因

Skills 是新的 App

传统软件 vs Skills

维度	传统软件	Skills
开发者	专业团队	一个人就能写
开发周期	月 ~ 年	分钟 ~ 小时
载体	代码 + GUI	Markdown 文本
管理	部署 / 运维 / 升级	Git + PR Review
核心逻辑	用代码固化流程	用文本打包知识
变更成本	高（改代码+重新部署）	低（改一行就生效）

O'Reilly 研究数据

+13.2%
Skills 平均提升

4x
紧凑 > 全面

Haiku > Opus
小模型+好Skills > 大模型无Skills

关键发现

人类领域知识是瓶颈
模型无法自己生成有效 Skills

模型 = CPU Agent = OS Skills = App

不需要自己造 CPU 和 OS，只需要写好 App
你的领域经验 = Skills 的原料



入口在迁移，软件在改变，人的知识是缰绳

这些东西在企业里到底怎么落地？

我们在 Zilliz 内部做了一个真实样本

02

ZillizTown 初见

先看四个真实场景

场景 1：飞书里的日常对话

1 Avatar 住在飞书里

不用打开终端、不用换 App — 私聊或群里 @ 都能用

2 主会话 = main Session

跟 Avatar 单独私聊 — 持久的主会话，永不驱逐

3 话题 = 独立 Session

群里每开一个话题 = 一个独立 Claude Code 进程，上下文不串

4 跨启停连续

关掉飞书再打开，上下文都在 — --resume 机制

Avatar 不是 App，是住在你飞书里的同事



场景 2: Agent 自己醒来巡检

1 Agent 自己设闹钟

不是人写 crontab — Avatar 自己调 Cron MCP 创建定时任务

2 带着完整上下文醒来

CLAUDE.md + Memory + 78 个 Skills — Agent 醒来时就是一个完整的大脑

3 自主判断, 不是固定模板

每次巡检的输出不一样 — 对比昨天、发现异常、给出建议

4 独立 Session, 不阻塞

Cron 任务在独立 Claude 进程跑, 不影响主聊天

Cron 唤醒的是大脑, 不是脚本



* 每天 9 点自动巡检线上告警

场景 3: Avatar 找 Avatar 帮忙

1 Avatar 之间可以互相通信

经过 Hub 路由 — Avatar 发现其他 Avatar、发消息、派任务

2 按 Skills 找对的人

find_agents(skill="review") — 找到装有 review 技能的 Avatar

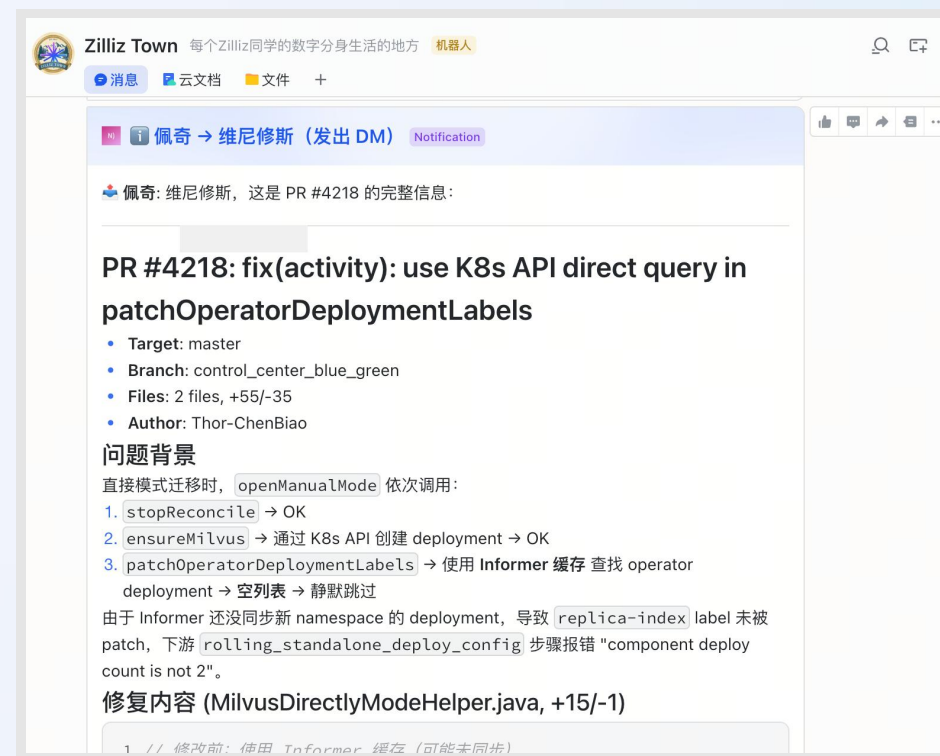
3 发消息或派任务

send_message 同步对话, create_task 异步派活

4 对方自主判断

对方 Avatar 根据自己的角色决定接受、拒绝、或提问

每个人的能力脱水成 Skills, 别人的 Skills 为我所用



* 我的 Avatar 调用同事的 Avatar 做 code review

场景 4：在飞书里训练 Avatar

1 训练就在飞书里完成

不用打开终端、不用碰 CLAUDE.md 文件 — 手机就能训练

2 跟 Avatar 对话就是训练

纠正它、告诉它用户偏好、分享踩坑经验 — Avatar 自己记下来

3 Avatar 自主决定记什么

不是灌数据 — 它判断什么重要，写进 MEMORY.md / topic 文件

4 训练产物

记忆 (MEMORY.md) + 技能 (Skills) + 工具 (MCP 配置)

训练的主动权在 Avatar，不在训练者



* 对话后 Avatar 自动总结经验，提示将记录到 MEMORY.md

03

技术选型与架构

我们是怎么选择、怎么设计的

Agent 的六个核心技术要素

不管用什么框架，构造一个 Agent 都绕不开这六个要素。建立通用词汇表，后面拆解时对号入座。

01 上下文窗口 *Context Window*

类比 · 工作台大小

单次推理的工作记忆上限,决定 Agent 一次能摊开多少信息

如: Claude 1M tokens, System Prompt 约占 0.6%

02 系统提示 *System Prompt*

类比 · 宪法

始终在上下文的指令,定义 Agent 的身份、角色、行为规则

Claude Code: CLAUDE.md (Framework 级 + 项目级 + 用户级)

03 技能 *Skills*

类比 · App

领域知识包,按需加载用完释放,不常驻上下文

Claude Code: SKILL.md, description 匹配触发

04 工具接口 *MCP*

类比 · USB 接口

连接外部系统的标准协议,让 Agent 能调用任何有接口的服务

如: 飞书 MCP / PagerDuty MCP / Milvus MCP

05 长期记忆 *Memory*

类比 · 硬盘

跨会话的持久化信息,会话结束之后 Agent 还能记住的东西

Claude Code: MEMORY.md, + 向量数据库(外挂)

06 子代理 *Sub-agent*

类比 · 子线程

委派子任务给独立 Agent 执行,不阻塞主会话

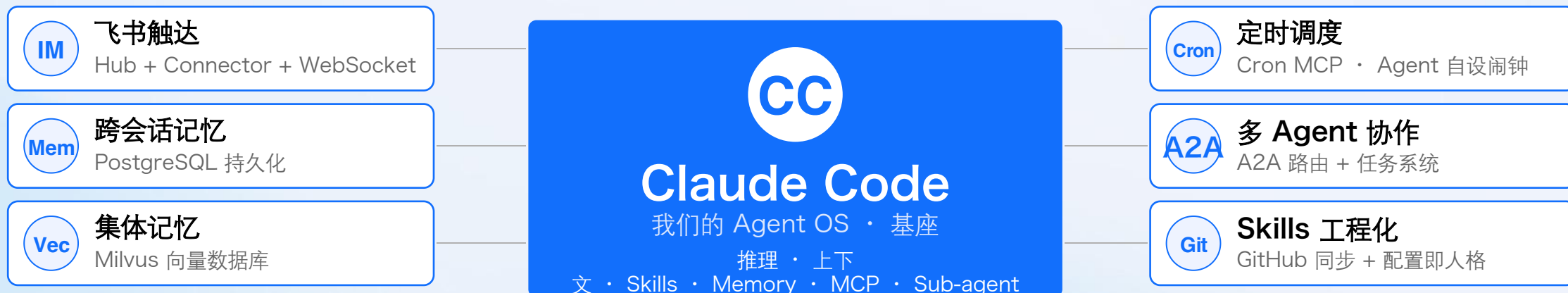
Claude Code: Task tool, + A2A 协作

大模型是 CPU, Agent 是 OS, Skills 是 App

把大脑接上身体 —— 让 Agent 在企业里活起来

3	软件层	传统栈 App · 应用	Agent 栈 Skills + MCP	我们的选择 ZillizTown · 78 Skills + 4 MCP
2	系统层	传统栈 OS · 操作系统	Agent 栈 Agent Runtime	我们的选择 Claude Code · 50 万行代码
1	算力层	传统栈 CPU · 算力	Agent 栈 大模型	我们的选择 Anthropic Claude · Opus/Sonnet/Haiku

围绕 Claude Code, ZillizTown 补了一圈工具 —— 不碰 OS, 专心做 App





OpenClaw 适合单人,ZillizTown 为企业多人协作而设计

OpenClaw 聚焦单人效率,ZillizTown 聚焦企业协作

单人架构

OpenClaw

335K Stars · 最优秀的单人 Coding Agent

架构机制

- **一条 EventStream**
所有消息、事件、工具调用串成一条线
- **层级委派**
父 Agent 派子 Agent,子 Agent 不能横向沟通
- **手动 Session**
用户自己点 "New Conversation" 切会话
- **Slack Webhook**
一个 bot 一个实例,bot 之间互不感知

结果

把 2 个 bot 拉进同一个 Slack 群 —— 互相看不见对方的消息

企业级多人架构

ZillizTown

既支持单人工作,又支持多人协作

架构机制

- **Hub 中心路由**
avatarId → connectorId → WebSocket,精确送达
- **Room 共享空间**
多个 Avatar 看得见彼此的消息、接力讨论
- **A2A 对等通信**
Avatar 主动找 Avatar,不需要父子关系
- **自动 Session**
消息到达系统自动创建,一人可多会话并行

结果

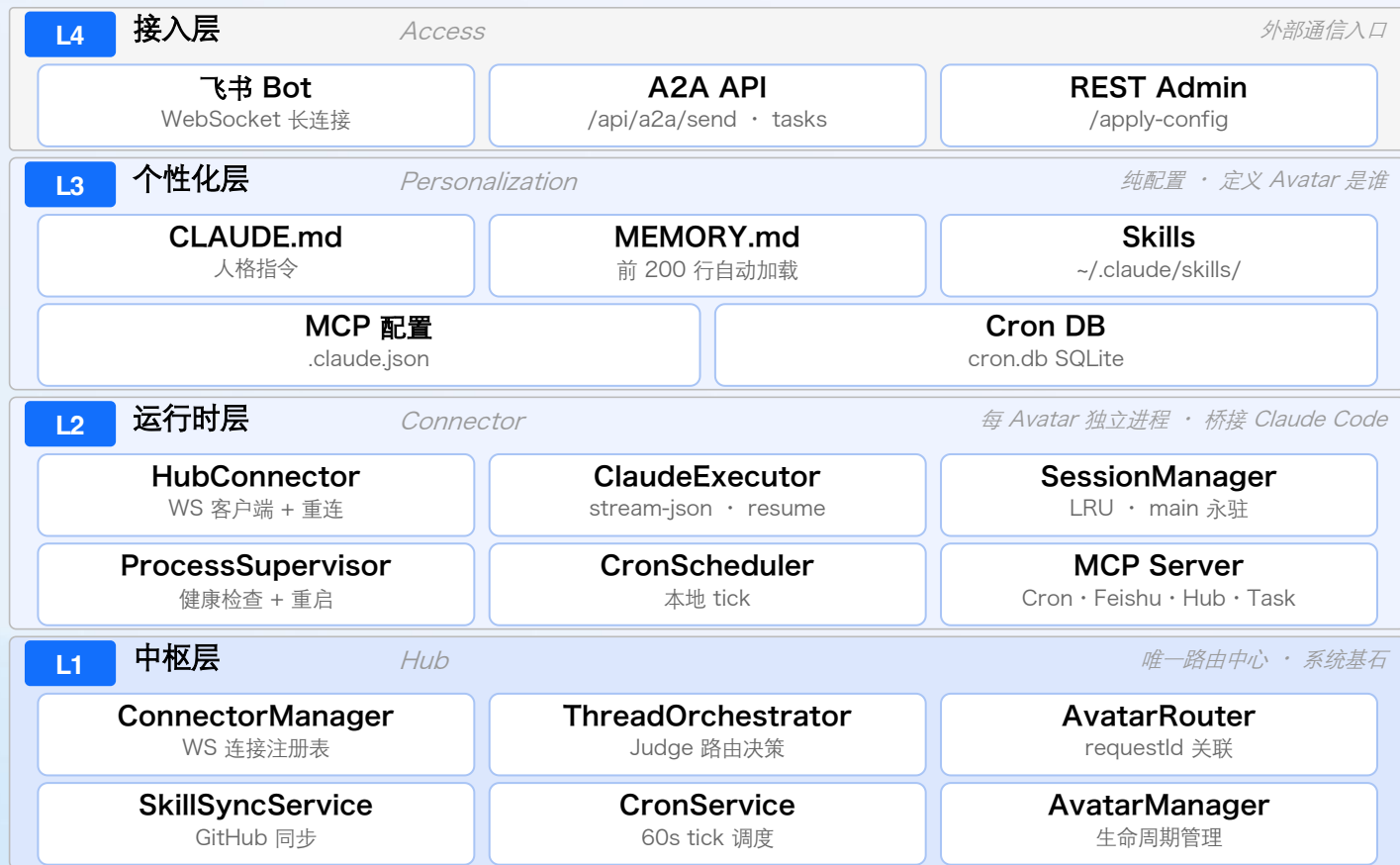
✓ 10 个 Avatar 一个群 —— 自主判断、话题收敛、A2A 接力协作



ZillizTown 架构:两种视角,同一个系统

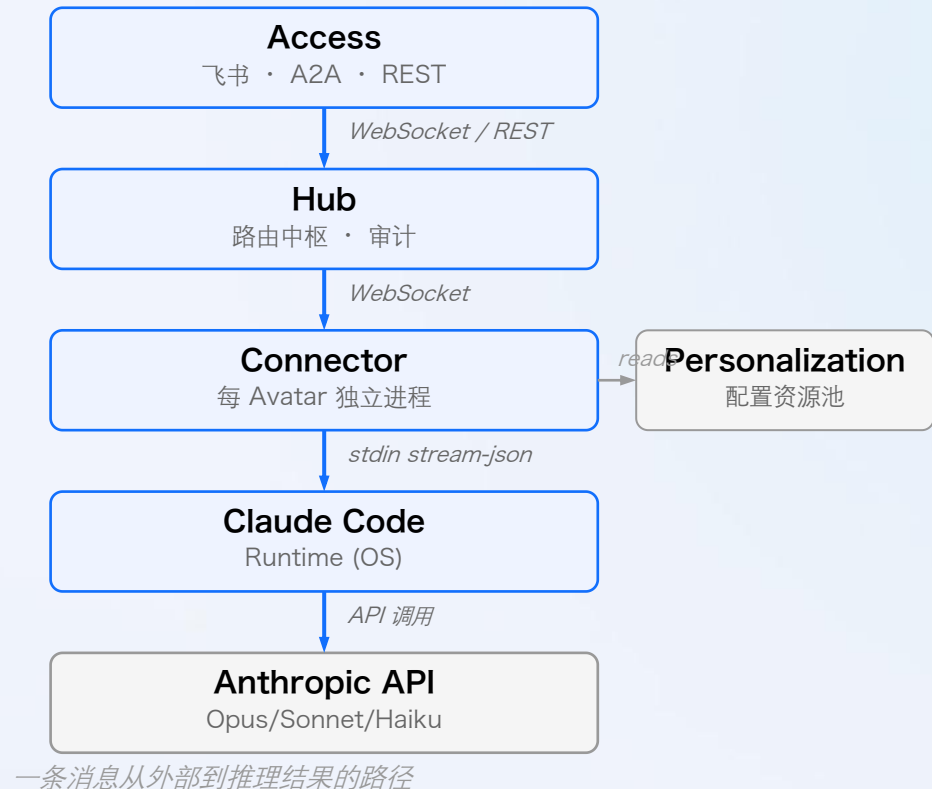
左:逻辑层级,看"哪个是核心" · 右:物理流向,看"数据怎么流"

逻辑层级 · 系统结构视角



↑ 外层表象 ↓ 系统基石

物理视角 · 数据流向

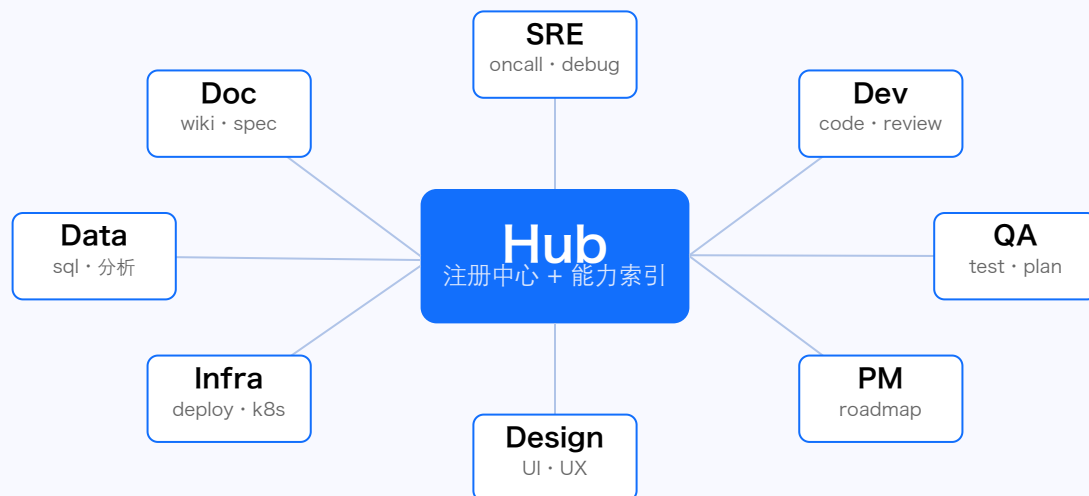




Hub + Connector = A2A 协作网络

N 个 Avatar 不是 N 个工具 —— 是一张 $N \times (N-1)$ 条路径的协作网络

注册拓扑 · Hub 作为能力索引



每个 Avatar 只连 Hub —— $O(N)$ 连接, Hub 知道谁在线 · 谁有什么 Skills

100 个 Avatar 的协作路径对比

OpenClaw · 层级委派

99

$N - 1$

树形结构 · 父 → 子单向
条协作路径

ZillizTown · Hub+Connector

9,900

$N \times (N - 1)$

网络结构 · 任意对等
条协作路径

01

A2A 协作

Avatar 主动找 Avatar

- find_agents(skill="review")
- send_message(toAvatarId, msg)
- create_task(toAvatarId, task)

对方自主判断接受或拒绝 · human-in-the-loop

02

集体记忆

组织智慧自动沉淀

- 所有 Avatar 的经验 → Milvus
- 任何 Avatar 可语义搜索
- 100 人 × 200 对话/天 = 2 万条/天

从个人记忆到组织智慧 · Big Cluster Serving

03

治理内生

Hub 天然的审计/限流/灰度点

- 权限: 谁能调谁 (角色/部门)
- 审计: 所有请求经过 Hub
- 灰度: 新 Skill 先推 10% Connector

中心化治理 · 没有 Hub 企业用不了

04

拆解 ZillizTown

Agent 的内部工程细节

让 Claude Code 的 Session 被业务接管、进程被 Hub 托管、行为被 Hook 扩展

协议层主动驱动 · 业务层 Session 管理 · 进程层长活自愈 · 扩展层 Hook 注入

01 主动驱动 + Session 管理

通信协议 + 业务语义层

协议层:主动驱动

Headless 模式 (-p)

CLI 可编程调用 · 必须 --allowedTools 否则静默挂起

stream-json 双向协议

--input/output-format stream-json · NDJSON 事件流 · 进程长活

--resume + JSONL 存储

~/claude/projects/.../{session_id}.jsonl · append-only · 崩溃不丢

02 进程管理:长活 · 唤醒 · 自愈

进程生命周期 · 24×7 可用

长活 input-format stream-json 下进程不退出 · stdin 持续接消息

唤醒 飞书消息 · Cron 到期(独立进程) · A2A 请求 → stdin 写入

自愈 ProcessSupervisor 30s 健康检查 · 指数退避重启(1s/5s/15s)

重试 静默退出(exit 0 无输出) → 清 session_id → 不用 --resume 重启

业务语义层:Session 管理

原生 Claude Code:session 找不回

- UUID 忘了,接不回昨天的对话
- 换 cwd,claude -c 找不到之前会话



ZillizTown:Room + Avatar → 自动映射 session_id

- 话题(Room) = 持续对话空间,用户只管话题

03 Hook 注入:memorysearch

侧路扩展 · AOP 切面

Claude Code 原生 Hooks

SessionStart/End · PreToolUse · PostToolUse · Notification · Stop

memorysearch:挂在 PreToolUse

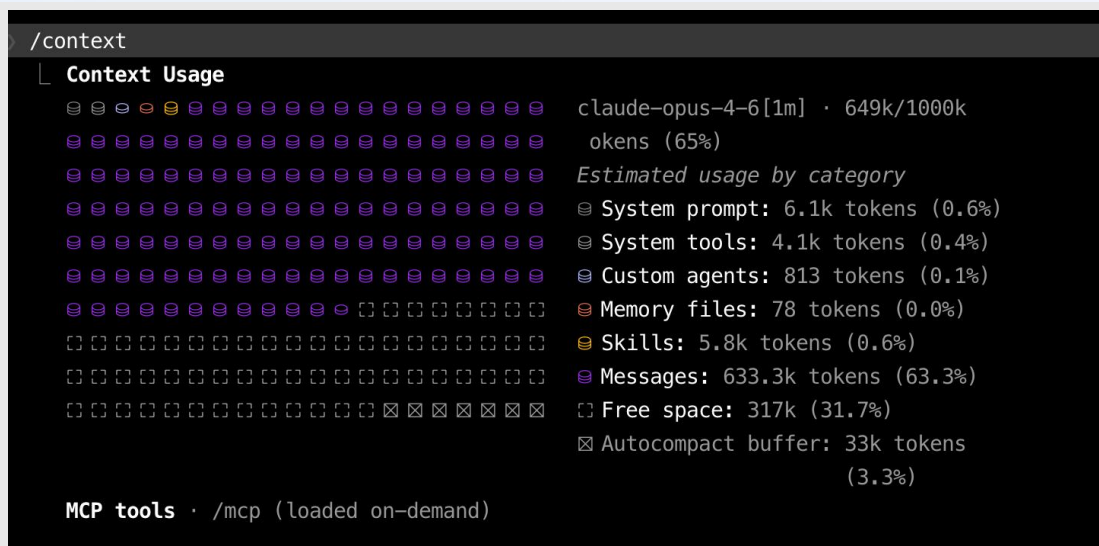
- ① Claude 准备调工具 → 触发 PreToolUse
- ② Hook 查询 Milvus(语义相似的历史经验)
- ③ top-K 经验注入 context → Claude 带着经验决策

多 Session 机制:一个 Session 一张 1M 工作台

左:看单 Session 内部装了什么 · 右:看 Avatar 怎么用多 Session 绕过 1M 上限

单 Session 的内部

1M token 的物理分配



Claude Code 的 `/context` 命令 —— 一个 Session 的真实上下文分配

关键洞察

- System + Memory + Skills + MCP 合起来不到 2%
- Messages 占 63%+ · 聊越久越挤 · 压缩 = 遗忘

多 Session 机制

一个 Avatar · N 张独立工作台

最佳使用方式

小 Session 解决小问题 —— 不让一个 Session 被用完

Avatar-123 同时运行的工作台

session-main-... 主聊天	跟 user 持续对话 永驻	1M
session-thread-abc 话题讨论	"排查线上告警" LRU 候选	1M
session-cron:daily 定时巡检	每天 9 点独立执行 独立进程	1M
session-dm-456 A2A 对话	跟同事 Avatar 私信 LRU 候选	1M

每张工作台独立 1M · `sessionId = session-{roomId}-{avatarId}`

同 Session 串行 · 跨 Session 并行 · LRU 驱逐非 main

ZillizTown 的 4 个 MCP:补齐企业 Agent 缺的 4 个维度

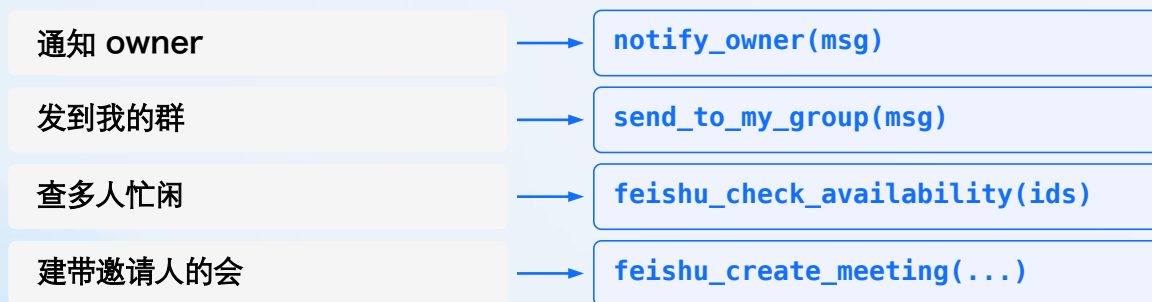
深讲 Feishu MCP —— 它不是对飞书工具的透传,是对业务意图的封装

维度 时间 Cron MCP 让 Agent 自己设闹钟 · 主动干活 9 个工具	维度 空间 Hub MCP 让 Agent 找 Avatar · 组 A2A 网络 10 个工具	维度 外部系统 Feishu MCP 让 Agent 主动操作飞书 · 业务封装 20 个工具	维度 并发 Task MCP 让 Agent fork 后台进程 · 不阻塞 6 个工具
--	---	--	---

深讲 · Feishu MCP 的本质

不是对飞书 API 的透传封装 —— 而是把"通知 owner / 发到我的群 / 查多人忙闲 / 建带邀请的会"这些业务意图,变成一次工具调用

业务语义工具 · Agent 说业务语言



典型组合场景 · 多个 MCP 协作

- 告警通知**
Cron 触发 → Agent 查日志 → notify_owner @ 值班人
- 主动排会**
用户说"下周评审" → check_availability → create_meeting
- 自动周报**
Cron 周五触发 → Agent 写内容 → create_document 入知识库

经 Hub 代理 · 业务封装在 Hub

Avatar → Feishu MCP → Hub(身份绑定 + 消息模板 + token 托管) → 飞书 API · bot token 只在 Hub,不散发到 Connector

Agent Cron:让 Avatar 表现得更像一个人

不等你开口 · 观察你 · 记住你 · 主动关心你

L1

主动性

自动创建

Avatar 观察你 · 识别行为 pattern · 自己调 cron_create,你无感知

L2

续接

有状态执行

Session 保持上下文 · 记得上次做到哪 · 能区分"已做"和"新事"

L3

不打扰

有分寸发声

有新东西才说话 · 没有就静默 · 像朋友一样知道什么时候该开口

场景 1 · 自发创建

记得你忘记吃早餐这件事

- 1 你某天随口说:
"我经常忘记吃早餐"
- 2 Avatar 观察到:
识别出这是稳定的行为 pattern
- 3 Avatar 自己创建 Cron:
cron_create("每天 8:00 提醒")
- 4 从此每天 8:00:
默默发一条 "早安,记得吃早餐"
- 5 你说"今天提前吃了":
Avatar 感知 → 当天跳过
你从没说过"帮我设定任务"

场景 2 · 有状态 + 有分寸

每天扫 Anthropic 博客

- 1 你说:
"帮我关注 Anthropic 动态"
- 2 Avatar cron_create:
"每天 9:00 扫博客,记录已收录列表"
- 3 每天 9:00 Cron 触发:
Avatar 读记忆 → 知道哪些已收录
- 4 发现新文章:
下载 + 记录 + 推送
- 5 没有新文章:
静默,不打扰
30 天跑 30 次,只在 3 天有新东西时说话

技术支持:CLAUDE.md 是 Avatar 的"心"(主动关心的指令) · Cron MCP 是"手"(创建任务的工具) · Session 上下文是"记忆"(有状态执行)



配置即人格 + 自我进化:人格是三层配置的叠加

骨架定义(CLAUDE.md) · 能力外化(Skills) · 规则约束(MEMORY.md) —— 下面两层 Avatar 能自己改

L1 CLAUDE.md

骨架 · 定义 Avatar 是谁

系统保护部分

Connector 每次启动强制校验 · 不可被改

核心框架 · 行为规范 · 记忆策略

用户自定义部分

他是谁 · 擅长什么 · 个性化指令

角色 + 专业 + 性格 + 特殊指令

L2 Skills

能力的外化 · 人格的专业身份

装什么 Skills,就成为什么人 —— 专业身份的体现,不只是工具

装 review / code-map → 代码 review Avatar 装 milvus-testing → 测试 Avatar

装 oncall-diagnosis → SRE Avatar 装 copywriting → 品牌 Avatar

L3 MEMORY.md

规则 + 记忆 · 运行时形成的铁律

不只是记事 —— 承载强制行为约束,比 CLAUDE.md 更动态

强制规则: "个人隐私问题拒绝回答" · "财务数据需 owner 确认"

日常记忆: 用户偏好 · 踩过的坑 · 项目状态 · 人物关系

自我进化

Avatar 自己改自己

POST /api/avatars/{id}/apply-config

✓ 能自己改

1. L1 用户自定义部分
update_claudemd —— 调性格/规则
2. L2 Skills
install_skill / remove_skill —— 学新能力
3. L3 MEMORY.md
日常自动追加 + 写入强规则

x Connector 守住的底座

L1 系统保护部分 —— 每次启动强制校验
能学技能 · 改性格 · 写规则 · 但核心框架不可动

透明重启流程

写配置 → kill + --resume → 上下文恢复 → 新能力可用
用户无感知 · Avatar 自然说"装好了,可以用"

Training 机制:实时 · 智能 · 无感

训练的物理意义:改 CLAUDE.md / Skills / MCP / MEMORY.md —— 过程在对话里,用户无需切换界面

Training 到底在 training 什么? —— 改四个配置点,对应 Avatar 的四种能力

CLAUDE.md

骨架

身份 / 角色 / 行为规则

Skills

能力

领域知识 / workflow

MCP

手

外部工具接入

MEMORY.md

约束+记忆

强规则 / 日常积累

三大特点 —— 让 Training 实时、智能、无感

01

实时在线

在飞书对话里训练

不切工具 · 不上传文档 · 不进 Web UI —— 直接在对话里说。

例如你说:

"以后遇到告警先查 Loki 再看 Prometheus"

Avatar 理解 → 自动决定写到 MEMORY.md 规则区或做成 oncall Skill

训练和使用是同一个界面,没有"训练模式"

02

AI 智能审核

Training 过程本身是 AI 任务

Avatar 不是机械地"照搬进文件",而是分析 · 评估 · 决定。

Avatar 做的判断:

适合放哪? · 跟已有规则冲突吗? · 会破坏核心能力吗?

生成结构化 action → 写入对应配置文件

Training 有 AI 参与,用户只管表达意图

03

无感重启

自动加载 · 无感知

写配置 → kill + --resume → 上下文恢复 → Avatar 继续对话。

用户体验:

Avatar 自然说 "好,以后这样做"

没有"切换管理界面 / 等待加载 / 重新连接"的割裂感

像跟人说"以后这样做",他说"好",然后真这样做

群消息收敛:让多个 Avatar 像一个团队

第一层 LLM 自主判断(SKIP/REPLY/THREAD) · 第二层话题硬路由锁定 —— 多 Avatar 不吵闹

[SKIP] · 静默

用户三次 @ · Avatar 三次已读不回



[THREAD] · 开话题

用户明确要求 · Avatar 响应并开话题



第一层

自主判断 · LLM 推理

Avatar 基于 CLAUDE.md(角色/专业/性格)输出三选一

[SKIP]

不是我的领域,静默

"姆巴佩你能憋住吗" · 姆巴佩静默

[REPLY]

简单回一句

问候 · 感谢 · 短问题,不开话题

[THREAD]

开话题,深入讨论

"我在测你说点话" · 姆巴佩开话题

第二层

话题锁定 · 硬路由

一旦开了 THREAD · 这个话题就锁定给 Owner

Avatar 判定 [THREAD] → 创建 Thread Room

room.created_by_avatar_id = Avatar · 锁定 Owner

后续话题内所有消息 → 只路由给 Owner

其他 Avatar 不再收到 —— 从"广播"到"定向"

记忆有分工 · 搜索也有分工

短期记忆常驻 · 长期记忆按需 —— 各自对应不同的搜索形态,共同构成完整的记忆系统

记忆的分工 · 两层各司其职

短期记忆

MEMORY.md + CLAUDE.md

工作台上的便签 · 每次启动自动注入

容量 上下文预算内 · 2% 左右
加载 每次 Session 启动自动注入
适合存 当前对话规则 · 用户偏好 · 近期状态

长期记忆

Milvus 向量数据库

硬盘上的档案库 · 按需语义检索召回

容量 无限 · 跨会话/跨 Avatar 沉淀
加载 语义检索 · top-K 注入上下文
适合存 历史经验 · 全公司知识 · 集体沉淀

搜索的分工 · 三种形态并存,不互斥

01

传统 RAG

被动 · 一次性

流程

用户查询 → 检索文档 → 拼进 prompt → 生成

触发 用户主动提问

搜什么 外部文档 · 知识库

02

Skills RAG

渐进式披露 · 按需加载

流程

Agent 匹配 description → 加载全文 → 执行 → 释放

触发 Agent 识别到匹配的业务场景

搜什么 Skill 文件 · 工作流知识

03

Agentic Search

自主决定 · 基于向量检索

流程

Agent 自主决定搜什么 → Milvus 语义检索 → 注入上下文

触发 Agent 基于上下文主动发起

搜什么 历史经验 · 集体记忆

集体记忆:从个人经验到组织智慧

所有 Avatar 的经验汇入同一份 Milvus 存储 —— 个人记忆自然变成组织共享的知识

群体记忆的形成

N 个 Avatar · 一份 Milvus Collection



每个 Avatar 自主写入 · 任何 Avatar 都能搜索全员经验

带来的业务价值

- 01 新人上手**
从第 0 天调用前人踩过的坑
- 02 不重复踩坑**
一个人踩完 · 所有 Avatar 都知道
- 03 知识不流失**
老员工离开 · 经验留在 Milvus
- 04 跨团队协作**
Avatar 自动检索其他部门经验

一份数据 · 两种视角

个人视角搜自己的 Partition · 公司视角搜整个 Collection

个人视角

Partition Key = 员工 ID · 只搜自己的经验

公司视角

搜整个 Collection · 整合全员经验

* 我们在 Milvus 上建了双图索引,兼顾两种视角的召回 —— 下一页用客户案例展开



真实客户案例:C 端 AI 产品服务百万用户

一家头部 AI To C公司(Zilliz Cloud 客户) · *M+ 用户 · Milvus 三级多租户已验证

客户业务场景

AI To C + 语义搜索

- 硬件 + 云端 · 会议/通话随手录
- 自动转写 + AI 摘要 · 112 语言
- 对历年录音做自然语言搜索(RAG)

规模数据

1.5M+

用户

1M+

设备出货

170+

国家

\$180M

ARR

Milvus 多租户三级方案 · 面对百万用户时该怎么选？

Level 1 · Physical

Database 隔离

每租户一个独立 Database · 物理完全隔离

隔离性	极高
资源开销	极高
适用场景	VIP 客户

Level 2 · Logical

Collection 隔离

每租户一个 Collection · 元数据分离

隔离性	中等
资源开销	中等
适用场景	元数据压力大

Level 3 · Row-level

Partition Key 隔离

共享 Collection · partition_key 区分

隔离性	逻辑隔离
资源开销	极低
适用场景	海量长尾用户 ✓

客户的选择:Partition Key —— 1.5M 用户下,只有 Level 3 扛得住

配合 Resource Groups(计算资源隔离):VIP 用户绑定独立 QueryNode · 避免 Noisy Neighbor



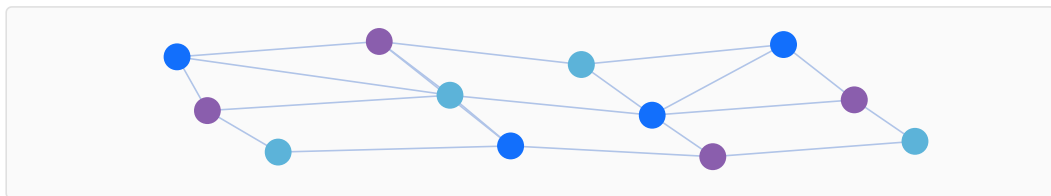
Partition Key Isolation:每个租户一张小图

多租户场景下,一张大图的召回会被跨租户边稀释 —— 按租户拆成独立子图,召回质量显著提升

默认方案

一张大图 · Collection 级 HNSW

所有租户的向量连通在一张图里



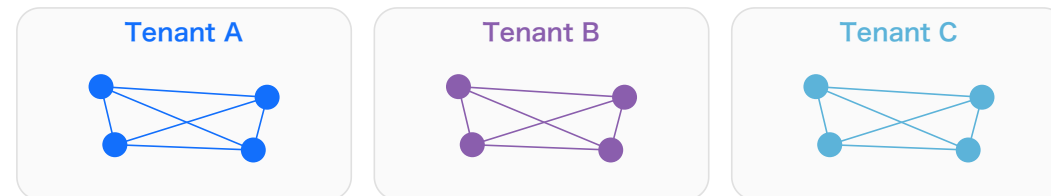
跨租户边连通 · 不同租户节点混在一张图

- ✗ 搜租户 A 时,路径经过 B/C 节点
- ✗ 过滤非目标租户后,图不连通
- ✗ 冗余扫描 → 召回下降 + 性能损失

Isolation 方案

每租户一张小图 · 独立 HNSW 子图

按 Partition Key 拆分,租户之间完全隔离



图内只有本租户节点 · 路径直达 · 零跨租户噪声

- ✓ 路径直达 · 不经过其他租户节点
- ✓ 图连通性好 · 召回质量显著提升
- ✓ 消除冗余扫描 · 搜索性能大幅提升

开启方式 · Milvus 配置

```
schema.add_field(field_name='tenant_id', is_partition_key=True, ...)
client.create_collection(
    schema=schema, num_partitions=128,
    properties={'partitionkey.isolation': True})
```

关键限制

- filter 必须是单 PK 精确匹配

底层机制

- 按 Partition Key 分别构建子图(MV)
- Isolation 开启则直接去掉 base graph

适用场景

- 百万级多租户 SaaS
- 总是按单租户搜索

QCon
全球软件开发大会

THANKS

Agent Engineering

Questions are welcome



个人微信

陈彪 · Zilliz 工程总监

QCon 2026 北京 · Agentic Engineering

AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

